

Enhancing Control-Flow Graph Recovery: Resolving Indirect Jumps with SMT Solvers and Direct Dominator Analysis

Albert Cao



Abstract

Control-flow graph (CFG) recovery is a core technique in cybersecurity, allowing developers to glean the execution paths of a program from only the binary code, without the risk of running the program. Indirect jumps are elements of CFGs where it is not clear where a portion of a program will go to. The dominant strategy is to use lossy heuristics. We present a more accurate method for resolving indirect jumps, using satisfiable modulo theory (SMT) solvers and direct dominators. Our approach achieves 100% accuracy on small programs.

Background

Currently, popular tools for CFG recovery depend on heuristics to quickly resolve indirect jumps, which leads to lower accuracy. The CFG is constructed by drawing connections between parts of the program, called “blocks,” where a connection represents that a block could possibly “jump” to another block due to a conditional statement, function call, or other part of the control flow of the program. Indirect jumps happen when a block has an ambiguous connection, most commonly arising in object-oriented programming through virtual jump tables.

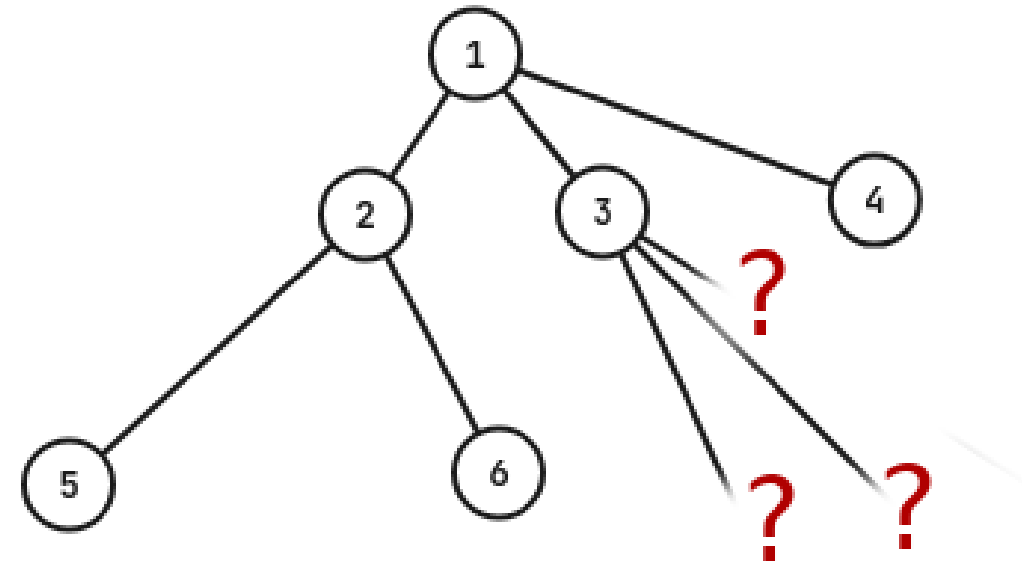


Figure 1: In the graph above, the incomplete connections represent indirect jumps from node 3.

SMT solvers are capable of determining solutions to variables based on a set of constraints. This is usually done through representing variables as bitvectors, and if the set of constraints is satisfiable, then a value is returned. In particular, as indirect jumps usually occur when a function call depends on a variable, by finding possible values for the variable we can also resolve the targets of the jump. We used yices, a state-of-the-art SMT solver in our project.

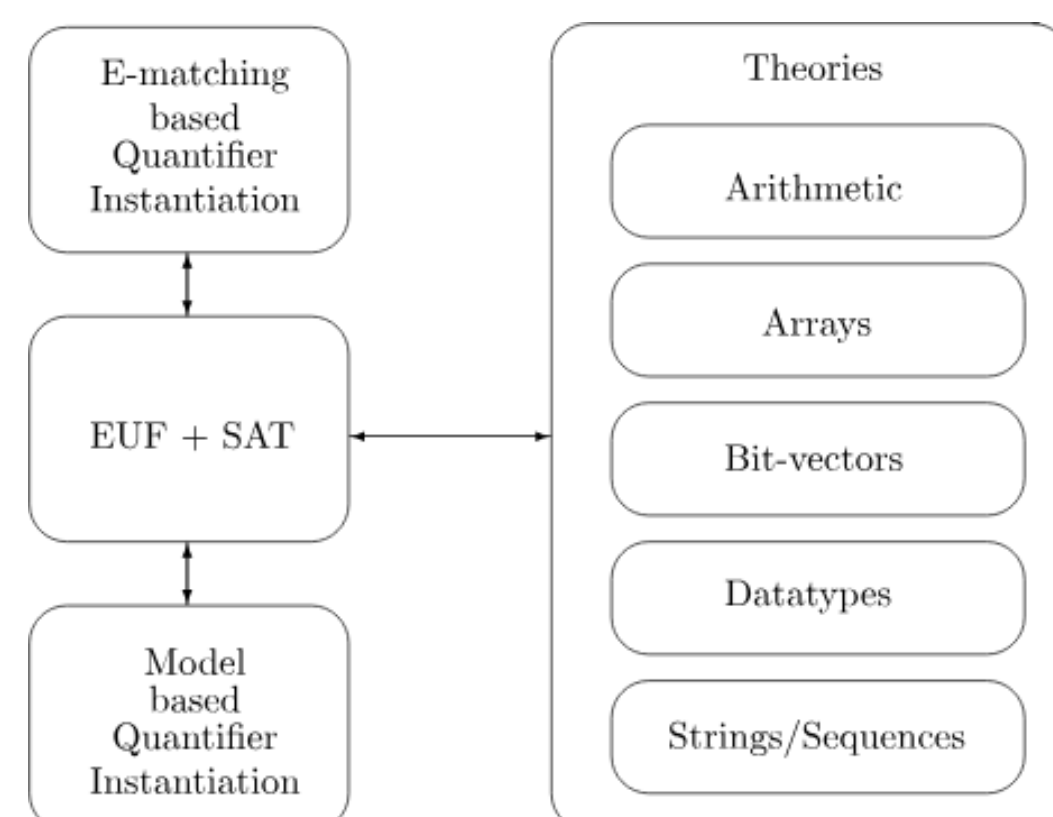


Figure 2: An architectural diagram of a similar SMT solver.

Direct Dominators

In a CFG, a direct dominator B of a block A satisfies that:

- All possible paths after B go through A,
- All possible paths to A go through B

Thus, in order for A to execute, B must have executed, implying that constraints in B are necessary for A.

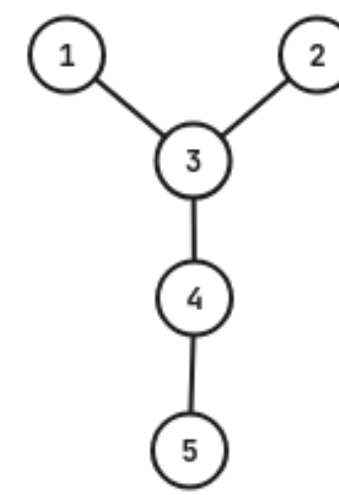


Figure 3: in the graph above, 3 and 4 are direct dominators of 5.

Methodology

We first lift the binary program into an intermediate representation known as P-Code, to easily identify the constraints represented by program commands. Thus, a block is a sequential list of P-Code, and P-Code contains an intermediate representation of instructions that are amenable to constraint synthesis. To better represent the relationships between constraints, we further use Abstract Data Graphs (ADGs), representing connections between constraints and the variables involved.

Similarly, each block also stores source blocks that could have came before, and destination blocks that could come after. Resolving indirect jumps resolves possible destination blocks.

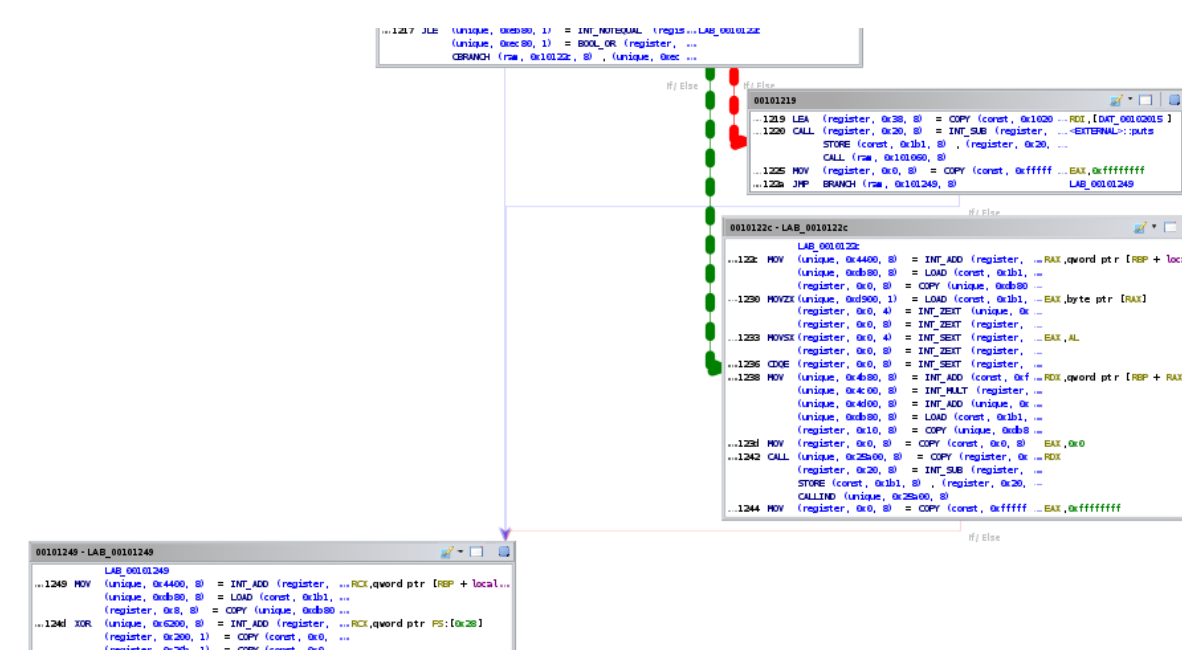


Figure 4: A CFG of blocks with P-Code included.

An indirect jump is identified when the P-Code of a block has an ambiguous target. Traversing up the CFG we can identify direct dominators, and add the P-Code of each block as ADGs to a model.

Once all the ADGs are collected, they are treated like constraints which are then solved for using yices. By repeatedly solving and adding constraints to ignore previous values, we may exhaust the possible values for the target of the indirect jump.

Results

We tested our approach on two programs. One, written in C, uses a virtual jump table, known to cause an indirect jump, and the other uses a switch table. In both cases, our approach correctly identifies the indirect targets. Furthermore, we also develop a program to visualize the results in a basic terminal output, and verify that the CFG is properly reconstructed.

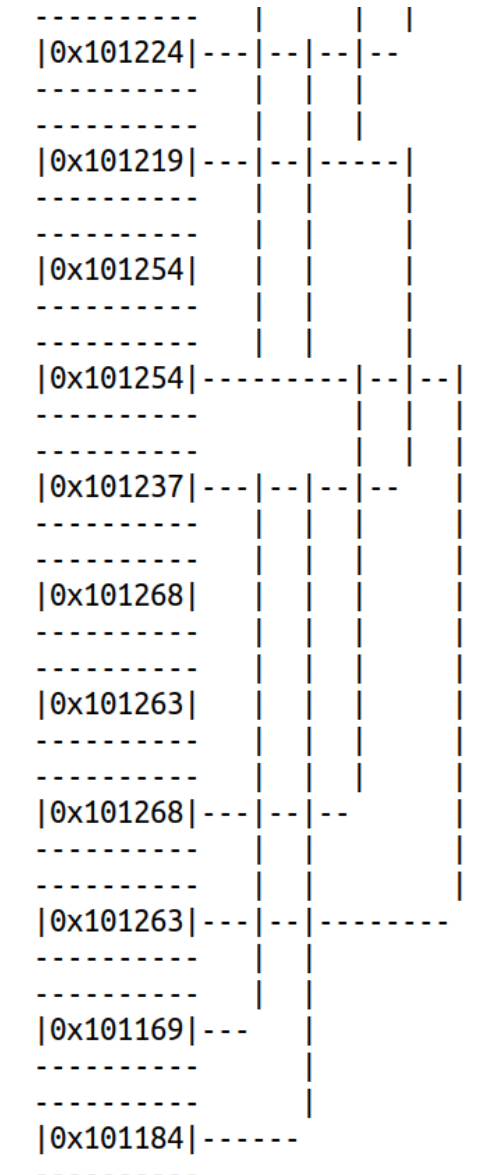


Figure 5: A portion of the completed CFG of a program.

Conclusion

We represented a binary program as P-Code, before constructing blocks and using ADGs to represent constraints of the program. Using the representation, we identified direct dominators and found indirect targets by using an SMT solver to analyze the constraints.

Our model proved to be highly accurate and efficient on small test cases. We hope to further improve the model to scale to larger programs, by more efficiently extracting values rather than running many solves. Furthermore, we hope to explore other possible heuristics to constrain the model without sacrificing accuracy. Potentially, the model can be used to assist developers to recover the CFG of a program accurately and quickly, aiding in cybersecurity by analyzing code safely.

Acknowledgments

I would like to thank my ASPIRE mentor Shane Donahue (AOS/QND) for his guidance throughout the project. Special thanks to Jessica Goodard (TSX/TAS) as the director of the ASPIRE program.

References

- Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Hauser, C., Kruegel, C., & Vigna, G. (2016). SoK: (State of) the art of war: Offensive techniques in binary analysis. In 2016 IEEE Symposium on Security and Privacy (pp. 138–157). <https://angr.io/>
- Spinsel. (2016). P-code reference manual. Retrieved April 27, 2025, from https://spinsel.dev/assets/2020-06-17-ghidra-brainfuck-processor-1/ghidra_docs/language_spec/html/pcoderef.html
- Björner, N., Eisenhofer, C., Gurfinkel, A., Lopes, N. P., de Moura, L., Nachmanson, L., & Wintersteiger, C. M. (2025). Z3 Internals (Draft). Microsoft Research. <https://z3prover.github.io/papers/z3internals.html>